Hashes also respond to `to_query` but with a different signature. If no argument is passed a call generates a sorted series of key/value assignments calling `to_query(key)` on its values. Then it joins the result with "&":

```
{c: 3, b: 2, a: 1}.to_query # => "a=1&b=2&c=3"
```

The method `Hash#to_query` accepts an optional namespace for the keys:

```
{id: 89, name: "John Smith"}.to_query('user')
# => "user%5Bid%5D=89&user%5Bname%5D=John+Smith"
```

Defined in `active_support/core_ext/object/to_query.rb`.

## 2.10 with_options

The method `with_options` provides a way to factor out common options in a series of method calls. Given a default options hash, `with_options` yields a proxy object to a block. Within the block, methods called on the proxy are forwarded to the receiver with their options merged. For example, you get rid of the duplication in:

```
class Account < ActiveRecord::Base
  has_many :customers, dependent: :destroy
  has_many :products,  dependent: :destroy
  has_many :invoices,  dependent: :destroy
  has_many :expenses,  dependent: :destroy
end
```

this way:

```
class Account < ActiveRecord::Base
  with_options dependent: :destroy do |assoc|
    assoc.has_many :customers
    assoc.has_many :products
    assoc.has_many :invoices
    assoc.has_many :expenses
  end
end
```

That idiom may convey *grouping* to the reader as well. For example, say you want to send a newsletter whose language depends on the user. Somewhere in the mailer you could group locale-dependent bits like this:

```
I18n.with_options locale: user.locale, scope: "newsletter" do |i18n|
  subject i18n.t :subject
  body    i18n.t :body, user_name: user.name
end
```

Since `with_options` forwards calls to its receiver they can be nested. Each nesting level will merge inherited defaults in addition to their own.

Defined in `active_support/core_ext/object/with_options.rb`.

## 2.11 JSON support

Active Support provides a better implementation of `to_json` than the `json` gem ordinarily provides for Ruby objects. This is because some classes, like `Hash`, `OrderedHash` and `Process::Status` need special handling in order to provide a proper JSON representation.

Defined in `active_support/core_ext/object/json.rb`.

## 2.12 Instance Variables

Active Support provides several methods to ease access to instance variables.

### 2.12.1 instance_values